Principles of Robot Autonomy I
Homework 2
Due Thursday, October 15 (11:59pm PT)

Starter code for this homework has been made available online through GitHub. To get started, download the code by running `git clone https://github.com/PrinciplesofRobotAutonomy/AA274a-HW2-F26` in a terminal window.

You will submit your homework to Gradescope in two parts:

1. **HW2 Code**: upload `P1_trajopt.py`, `P2_dynamics.py`, and `P3_lqr.py`. These are graded by an autograder (parts marked 🖥). Some tests run as soon as you submit, and the rest are released after the deadline, so passing the visible tests does not guarantee full credit. Before the deadline you will only see the results of the visible tests. You may resubmit as many times as you like; only your last submission counts. Do not rename the files or change the function signatures.
2. **HW2 Written**: a single pdf with your answers to the written questions and the requested plots (parts marked ✎). For short-answer questions, write your final answer on its own line starting with **Answer:**, followed by a brief justification where asked.

Your submission must be typeset in LaTeX.

You may discuss the problems with your classmates, but you must prepare your solutions on your own and list the names of the classmates you worked with. Using generative AI tools to substantially complete the assignment is not permitted, and any use beyond incidental use must be acknowledged in your submission. See the course outline for the full policy.

Introduction

The goal of this homework is to familiarize you with some Python fundamentals that will be used throughout the quarter, as well as techniques for computing trajectories with trajectory optimization and for tracking them with feedback control, for a differential drive robot and a quadrotor.

Nonholonomic Wheeled Robot

Throughout this homework, we will consider a robot that operates with the simplest nonholonomic wheeled robot model, the unicycle, shown below in Figure 1.

The *kinematic* model we will use reflects the rolling without side-slip constraint, and is given below in Eq. (1).

$$\begin{aligned} \dot{x}(t) &= v(t)\cos(\theta(t)), \\ \dot{y}(t) &= v(t)\sin(\theta(t)), \\ \dot{\theta}(t) &= \omega(t). \end{aligned} \tag{1}$$

In this model, the robot state is $\mathbf{x} = [x, y, \theta]^T$, where $[x, y]^T$ is the Cartesian location of the robot center and θ is its heading with respect to the x-axis. The robot control inputs are $\mathbf{u} = [v, \omega]^T$, where v is the velocity along the main axis of the robot and ω is the angular velocity.

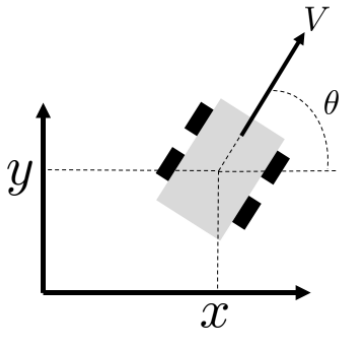


Figure 1: Unicycle robot model

Problem 1: Trajectory Optimization [22 Points]

Let's consider the problem of designing a dynamically feasible trajectory. In finding a trajectory that connects a starting state to the goal state in a dynamically feasible manner, there are often many such dynamically feasible trajectories, and we might prefer some to others. In this problem, we will utilize tools from optimal control to design a trajectory that explicitly optimizes a given objective.

Consider the kinematic model of the unicycle given in Eq. (1), where v is the linear velocity and ω is the angular velocity. Suppose the objective is to drive from one waypoint to the next waypoint with minimum time and energy, i.e., we want to minimize the functional

$$J = \int_0^{t_f} \left[\alpha + v(t)^2 + \omega(t)^2\right] dt,$$

where $\alpha \in \mathbb{R}_{>0}$ is a weighting factor and t_f is free. The controls are limited to $0.01 \leq v(t) \leq v_{\max} = 0.5$ and $|\omega(t)| \leq \omega_{\max} = 1$. We'll use the following initial and final conditions.

$$\begin{aligned} x(0) = 0, \quad y(0) = 0, \quad \theta(0) = \pi/2, \\ x(t_f) = 5, \quad y(t_f) = 5, \quad \theta(t_f) = \pi/2. \end{aligned}$$

In addition, we consider an object in the environment that the robot must avoid. We formulate this collision avoidance as follows:

$$\sqrt{(x(t) - x_{obstacle})^2 + (y(t) - y_{obstacle})^2} - (r_{ego} + r_{obstacle}) \geq 0,$$

where r_{ego} is the radius of the robot we control, and $r_{obstacle}$ is the radius of the obstacle to avoid. In this problem, we use

$$x_{obstacle} = 2.5, \quad y_{obstacle} = 2.5, \quad r_{obstacle} = 0.3, \quad r_{ego} = 0.1.$$

(i) ✏ (**7 Points**) Transcribe this optimal control problem into a finite dimensional constrained optimization problem, using N time intervals of equal length $\Delta t = t_f/N$ and forward Euler integration of the dynamics. State the decision variables, the function to be minimized, the initial and final conditions, the collision avoidance constraint, the dynamics constraints, and the bounds.

(ii) 💻 (**9 Points**) Implement the following functions in `P1_trajopt.py`, following your transcription from part (i):

- `cost`
- `dynamics_constraints`
- `boundary_constraints`

- `obstacle_constraints`

The decision variables are packed into a single vector `z` that holds t_f, the states $s_0, \ldots, s_N$, and the controls $u_0, \ldots, u_{N-1}$; use the provided function `unpack_decision_variables` to recover them. Each function must return an array of the shape given in its docstring. The provided `optimize_trajectory` passes your functions to `scipy.optimize.minimize`.[1]

Then open the notebook `P1_trajectory_optimization.ipynb` and run the "Sanity Checks" and "Trajectory Optimization" sections. If implemented correctly, your optimizer should produce a trajectory that reaches the goal without colliding with the obstacle. This part is graded by the autograder; you do not need to include anything in your write-up.

(iii) In this part we will predict how fast the optimal trajectory drives as a function of α.

(a) ✎ (**4 Points**) Suppose the robot drives along a straight segment of length L at a constant speed v with $\omega = 0$. Show that the cost of the segment is $J(v) = L\,(\alpha/v + v)$, and find the speed v^* that minimizes it subject to $0 < v \leq v_{\max}$. For which values of α is the speed limit active?

(b) ✎ (**2 Points**) Run the "Cruising Speed vs. α" section of the notebook and include the plot it saves, `plots/p1_alpha_sweep.png`, in your write-up. Report the median speed printed for $\alpha = 0.04$ and $\alpha = 0.25$, each next to your prediction from part (a).

Problem 2: Numpy and Class Inheritance [15 Points]

In this problem, we will demonstrate the use of class inheritance in Python and the use of Numpy for vectorized operations. Your code goes in `P2_dynamics.py`, and the notebook `P2_dynamics.ipynb` runs and plots it.

We will be using the `Dynamics` base class for two different dynamics models. The base class contains two unimplemented functions: `feed_forward` and `rollout`. The `feed_forward` function propagates the dynamics a single time step of length Δt with forward Euler integration and adds a random disturbance, and the `rollout` function applies `feed_forward` repeatedly to obtain a trajectory of states over multiple time steps. Because of the disturbances, the same control sequence results in different trajectories, which we will observe by executing multiple rollouts with the same control sequence from the same initial state. In both models, `rollout` returns an array of shape `(num_rollouts, num_steps + 1, n)`, where `n` is the state dimension.

The first model we consider is the kinematic model in Eq. (1). The second is the double integrator:

$$\begin{aligned} \dot{x}(t) &= v_x(t), \\ \dot{y}(t) &= v_y(t), \\ \dot{v}_x(t) &= a_x(t), \\ \dot{v}_y(t) &= a_y(t). \end{aligned} \tag{2}$$

In this model, the robot state is $\mathbf{x} = [x, y, v_x, v_y]^T$ and the robot control inputs are $\mathbf{u} = [a_x, a_y]^T$. Its Euler discretization is $\mathbf{x}_{t+1} = A\mathbf{x}_t + B\mathbf{u}_t$, with A and B given in the `DoubleIntegratorDynamics` class.

(i) ⌨ (**3 Points**) In the `TurtleBotDynamics` class, implement `feed_forward` using forward Euler integration of Eq. (1), and implement `rollout` with two `for`-loops that call `feed_forward`. Run the TurtleBot cells of the notebook to check your rollouts.

(ii) In the previous part we used a `for`-loop over rollouts. For the double integrator, we will instead step all rollouts at once by stacking them into a single vector $\mathbf{X}_t \in \mathbb{R}^{4R}$, where R is the number of rollouts.

[1] See https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.minimize.html.

(a) ✏ (**3 Points**) Write down the vectorized equation for a single dynamics step of all R rollouts (without the disturbance), using $\bar{A}$ for the matrix `A_stack` and $\bar{B}$ for the matrix `B_stack` built in the code. Give the dimensions of $\mathbf{X}_t$, $\bar{A}$, and $\bar{B}$, and write $\bar{A}$ and $\bar{B}$ in terms of A and B.

(b) 💻 (**4 Points**) In the `DoubleIntegratorDynamics` class, implement `feed_forward` without any `for`-loops, and implement `rollout` with a single `for`-loop over time steps (one call to `feed_forward` per step). Run the double integrator cells of the notebook to check your rollouts.

(iii) Forward Euler integration is only approximate. Consider the double integrator starting from rest at the origin with a constant acceleration $a_x = a$, so the exact position is $x(t) = at^2/2$.

(a) ✏ (**2 Points**) Run the "Euler Integration Error" cell of the notebook, which uses $a = 1$ m/s^2 and $T = 1$ s. Report the Euler position $x(T)$ for $\Delta t = 0.1$ s and $\Delta t = 0.01$ s.

(b) ✏ (**3 Points**) Show that after K Euler steps the position is

$$x_K = \frac{a\Delta t^2 K(K-1)}{2},$$

and that with $T = K\Delta t$ the error is $x(T) - x_K = aT\Delta t/2$. Check this against your numbers from part (a). In one or two sentences, explain why the Euler estimate falls short of the true position and how the error scales with Δt.

Problem 3: LQR with Gain Scheduling [33 Points]

In this problem, you will implement LQR with gain scheduling for a planar quadcopter (drone) that wants to reach a goal position while avoiding an obstacle in the presence of a wind disturbance.

Follow along the notebook `P3_gain_scheduled_LQR.ipynb`. Parts 1 through 3 of the notebook set up the dynamics (defined in `planar_quad.py`), load a nominal open-loop trajectory from `nominal_trajectory.npz`, and simulate it in the wind field stored in `wind_field.npz`; you do not have to write any code there. In Parts 4 and 5 you will use the functions you write in `P3_lqr.py` to track the nominal trajectory and compare a few variants.

Throughout, x_j^* and u_j^* denote the nominal state and control at step j of the nominal trajectory, $j = 0, \ldots, N$. Since there is no nominal control at the final state, u_N^* is taken to be the hover thrust (see the provided `nominal_control_at`). The LQR gain for a linear system $\dot{x} = Ax + Bu$ with weights Q and R is $K = R^{-1}B^TP$, where P solves the continuous-time algebraic Riccati equation

$$A^TP + PA - PBR^{-1}B^TP + Q = 0.$$

A gain-scheduled LQR controller linearizes the dynamics about each nominal point (x_j^*, u_j^*) to obtain (A_j, B_j) and a gain K_j, and applies $u = u_j^* - K_j(x - x_j^*)$ for a chosen index j.

(i) 💻 (**13 Points**) Implement the following functions in `P3_lqr.py`:

- `compute_lqr_gain`: the LQR gain K for given A, B, Q, R.
- `compute_gain_schedule`: the list of gains $K_0, \ldots, K_N$. Linearize with `get_continuous_jacobians` (in `planar_quad.py`).
- `lqr_control`: the control $u = u_j^* - K_j(x - x_j^*)$ for a given index j.
- `compute_hover_gain`: a single gain for the dynamics linearized about a level hover.
- `find_closest_nominal_state`: the index j of the nominal state closest to the current state, using the Euclidean norm of the full 6-dimensional state difference.

Run Parts 4 and 5 of the notebook. Part 4 schedules the gains by time, i.e., it uses $j = k$ at simulation step k. If implemented correctly, your drone should roughly follow the nominal trajectory and come close to the goal.

(ii) Short answers.

(a) ✎ (**2 Points**) What are the dimensions of each gain matrix K_j, and why?

(b) ✎ (**2 Points**) Which of the following makes the drone track the nominal trajectory more tightly? Give the letter and one sentence on what you give up in exchange. (A) Scaling Q up. (B) Scaling R up. (C) Scaling Q and R up by the same factor.

(c) ✎ (**4 Points**) Report the miss distance (the distance between the final position and the goal) printed in the notebook for: the open-loop controls, the time-indexed gain-scheduled LQR, the single hover LQR gain, and the nearest-state gain-scheduled LQR.

(iii) Discussion.

(a) ✎ (**6 Points**) Include `plots/p3_closed_loop.png` and `plots/p3_r100.png` in your write-up. Describe how the closed-loop behavior changes when R is multiplied by 100, and explain why in terms of the LQR cost $\int (x^T Q x + u^T R u)\, dt$.

(b) ✎ (**3 Points**) The single hover gain performs almost exactly as well as the full gain schedule on this trajectory. Using the Jacobians in `get_continuous_jacobians` and the maximum pitch printed in the notebook, explain why. Describe a maneuver for which gain scheduling would matter.

(c) ✎ (**3 Points**) Scheduling by the nearest nominal state fails here. Using the sequence of indices printed in the notebook, explain what happens to the drone and why it ends up hovering near its starting point. Suggest one change to the index lookup that would fix this.

Problem 4: ROS2 Navigation Node (Section Prep)

Note: This portion of the homework is **not graded**, but should be completed before your Week 6 section (10/26 - 10/30) to test in hardware.

Objective: Implement a Path Planning and Trajectory Tracking Node in ROS2 using A* Algorithm and Spline Interpolation

Important note: all the URLs are highlighted in blue. Make sure you click into them as they are important references and documentation!

In this assignment, you are tasked with developing a ROS2 node in Python that utilizes the A* algorithm for path planning and spline interpolation for trajectory generation and tracking for a TurtleBot3 robot. The node will be implemented using the `rclpy` library and will interact with custom messages and utility functions provided in the `asl_tb3_lib` and `asl_tb3_msgs` packages. You will be leveraging your implementations of A* and path smoothing from HW1, as well as the differential flatness tracking controller provided in `trajectory_tracking.py` in the HW2 starter code.

First, take a brief look at the `navigation.py` from `asl_tb3_lib`. Specifically, you will be implementing the functions `compute_heading_control`, `compute_trajectory_tracking_control`, and `compute_trajectory_plan`. In this file, you can also find the definition of the `TrajectoryPlan` class.

Unlike HW1, you will build your navigation node from scratch for this homework. However, feel free to use the given code for HW1 as a reference.

Implement the Navigation Node

Step 1: Create a new node. You can use the same autonomy workspace from HW1. In it, make a new script at `~/autonomy_ws/src/autonomy_repo/scripts/navigator.py`. Write the necessary code to create your own navigator node class by inheriting from `BaseNavigator`.

Hints:

1. Some examples for importing from `asl_tb3_lib`,

```
from asl_tb3_lib.navigation import BaseNavigator
from asl_tb3_lib.math_utils import wrap_angle
from asl_tb3_lib.tf_utils import quaternion_to_yaw
```

2. Use HW1, section, or this minimal node example as references on how to write the basic structure of a Python ROS2 node.
3. Make sure this script is a proper executable file (i.e. shebang + executable permission).
4. Register your new node in `CMakeLists.txt` at the root of your ROS2 package. See for example here.

Step 2: Implement / Override `compute_heading_control`. This should be identical to the function `compute_control_with_goal` from `heading_controller.py` in HW1. You may also want to add gain initialization to the `__init__` constructor.

Step 3: Implement / Override `compute_trajectory_tracking_control`. Migrate and re-structure the `compute_control` function in `trajectory_tracking.py`. This is not as straightforward as Step 2. Use the following hints as a guide:

1. Make sure to understand the data structures `TurtleBotControl` and `TrajectoryPlan`.
2. The desired states `x_d, xd_d, xdd_d, y_d, yd_d, ydd_d` need to be computed differently. Use `scipy.interpolate.splev` to sample from the spline parameters given by the `TrajectoryPlan` argument.

3. The variable initialization in the constructor (`__init__`) function also needs to be migrated. Constants like `V_PREV_THRESH` also needs to be moved into the constructor.

4. The control limit can be removed since the base navigator class has its built-in clipping logic to prevent generating unreasonably large control targets.

Step 4: Implement / Override `compute_trajectory_plan`. You will borrow / migrate code from the A* problem (HW1 Q1). You don't need to implement additional logic in this question, but you will need a solid understanding of `trajectory_tracking.py` in order to move things into the right places. The pseudo code for this function is detailed in Algorithm 1. Here are some hints for implementing each step of the algorithm:

1. Make sure you understand everything about the `AStar` class. The easiest way to implement this step is to copy the entire class into your navigator node, and directly use it in the `compute_trajectory_plan` method. See the notebook `sim_astar.ipynb` for examples on how to
   (a) construct an `AStar` problem
   (b) solve the problem
   (c) access the solution path
2. See `sim_astar.ipynb` for examples on how to check if a solution exists.
3. The `compute_trajectory_tracking_control` method uses some class properties to keep track of the ODE integration states. What are those variables? How should we reset them when a new plan is generated?
4. See `compute_smooth_plan` function from `sim_astar.ipynb`.
5. See the block below `compute_smooth_plan` on how to construct a `TrajectoryPlan`.

Algorithm 1 Compute Trajectory Plan

Require: `state`, `goal`, `occupancy`, `resolution`, `horizon`

Initialize A* problem using `horizon`, `state`, `goal`, `occupancy`, and `resolution` ▷ A* Path Planning
if A* problem is not solvable **or** length of path < 4 **then**
    return None
end if
Reset class variables for previous velocity and time ▷ Reset Tracking Controller History
Compute planned time stamps using constant velocity heuristics ▷ Path Time Computation
Generate cubic spline paramteres ▷ Trajectory Smoothing
return a new `TrajectoryPlan` including the path, spline parameters, and total duration of the path

Create the Launch File

Create a launch file at `~/autonomy_ws/src/autonomy_repo/launch/navigator.launch.py`. The launch file needs to

1. Declare a launch argument `use_sim_time` and make it defaults to `"true"`.
2. Launch the following nodes
   (a) Node `rviz_goal_relay.py` from package `asl_tb3_lib`. Set parameter `output_channel` to `/cmd_nav`.
   (b) Node `state_publisher.py` from package `asl_tb3_lib`.
   (c) Node `navigator.py` from package `autonomy_repo` (This is your navigator node!). Set parameter `use_sim_time` to the launch argument defined above.

3. Launch an existing launch file `rviz.launch.py` package `asl_tb3_sim` with the following launch arguments
   (a) Set `config` to the path of your `default.rviz`.
   (b) Set `use_sim_time` to the launch argument defined above.

Hint: take a look at `heading_control.launch.py` provided from HW1. You may copy the entire file over and make some really small changes to satisfy the requirements above. These requirements are mostly just descriptions of what the previously provided launch file is doing.